

Grounded question answering over compliance-training documents

Retrieval-augmented generation (RAG) knowledge system · code, data and full results:

github.com/praveeniitm/learning-compliance-rag

1. Problem and domain

Organizations that run a learning management system (LMS) must keep training assignments aligned with regulation. Compliance and safety teams ask questions such as "Is our annual HIPAA (Health Insurance Portability and Accountability Act) refresher required by law or only by our policy?" or "What rule applied when this employee was hired in 2025?". The answer sits across regulations, stricter versioned policies and the course catalog; errors mean wrongly assigned training and failed audits. The corpus combines 13 real regulation sections: 10 from the Occupational Safety and Health Administration (OSHA), 2 from HIPAA, and California's harassment-training law, fetched point-in-time from the electronic Code of Federal Regulations (eCFR). It adds 15 fictional internal documents, including a superseded policy, a wrong e-mail claim, a hidden prompt injection and role-restricted files. **Why RAG:** every answer must be traceable to a paragraph, and the rules change with each policy version. Fine-tuning would bake in rules that go stale and cannot cite sources, and a plain large language model (LLM) does not know internal policy.

2. Approach and algorithm decisions (alternative considered → why rejected)

- **Chunking: split at document structure, not at fixed length.** Markdown headings mark one regulation paragraph group, such as "(I) Operator training", or one policy section. Within a section, chunks are capped at 320 tokens with 32 tokens of overlap. 320 tokens holds a full paragraph group or clause list, and 5 chunks fit in about 2k prompt tokens; the overlap covers sentences cut inside long sections. *Lost at boundaries:* a child paragraph such as "(iii) ... at least once every three years" does not say it concerns forklift operators, and table rows lose their column names. *Mitigation:* each chunk starts with a context header holding the citation, title, everyday topic (e.g. "forklift", which the regulation never uses), section path and version/SUPERSEDED flag. Each regulation line keeps its own paragraph tag, e.g. [29 CFR 1910.178(l)(4)(iii)]. *Alternative:* fixed 320-token windows scored Recall@5 0.897 vs. 0.940 and mean reciprocal rank (MRR) 0.831 vs. 0.987, because they cut through tables and paragraph lists. Removing only the header lowered Recall@5 to 0.812. Semantic chunking adds cost and non-determinism to already-structured text.
- **Embeddings: text-embedding-3-small**, compared empirically on the test set in dense-only mode, where differences are not masked by re-ranking. text-embedding-3-large reached Recall@5 0.859 vs. 0.791, but the smaller, cheaper model inside the full hybrid pipeline reaches 0.940. The analytical reason: the hard cases are exact citations and codes, which keyword search and re-ranking solve better than a larger embedding does. Both the LLM and embeddings are OpenAI-hosted today. If data residency or high request volume makes that unsuitable, self-hosted open-weight models (e.g. Llama/Qwen, bge/e5 via vLLM) plug in through a base-URL setting; the index is rebuilt once and the evaluation re-run.
- **Retrieval, fusion and re-ranking.** BM25 keyword search uses a tokenizer that keeps "1910.178(l)(4)" and "EHS-FL-201" whole. FAISS (Facebook AI Similarity Search) runs dense vector search alongside it. Both lists are merged by reciprocal rank fusion (RRF), then an LLM re-ranks the top 20 fused candidates, of which the best 5 are kept. RRF uses ranks only, so unlike weighted score fusion it needs no calibration between BM25 and cosine scores. Over 39 test questions with gold passages:

One search, top 5	Precision@5	Recall@5	MRR
Dense only	0.292	0.791	0.751
BM25 only	0.236	0.688	0.692
Hybrid (RRF)	0.287	0.799	0.796
Hybrid + LLM re-rank (used)	0.344	0.940	0.987

Compared with dense search alone, fusion lifts MRR from 0.751 to 0.796 (the first correct passage appears higher) but leaves Recall@5 almost unchanged (0.791 → 0.799). BM25 and dense search each find answer passages the other misses, but fusion keeps only 5 of their 40 candidates: on some questions it gains a BM25-only passage, on others it drops one dense search had in its top 5, and these roughly offset. The re-ranker reads the query with each of the 20 fused candidates and keeps the best of both, lifting Recall@5 by 0.149 over dense alone. **Vector store:** FAISS with exact search; at 965 chunks a query takes under 1 ms with no server, and pgvector becomes worthwhile only with concurrent writers or database-enforced permissions.

- **Orchestration: a custom LangGraph agent** that only plans searches. It uses two tools: Search(query, source = regulation | internal | any) and Clarify(question). Law-vs-policy questions need a separate regulation search, because in one search internal documents out-rank the regulation they paraphrase; multi-part questions need several lookups. *Alternatives:* a fixed single-search pipeline can do neither, and a prebuilt ReAct (reason-and-act) agent would also write the answer, bypassing the output schema and grounding check. Control stays in code: the first step must call a tool, role and as-of date are applied inside the search, repeated queries are refused, and there are at most 3 searches.
- **Hallucination prevention and detection, four layers.** (1) The prompt allows facts only from numbered sources, requires a citation on every sentence, and marks sources as data, not instructions. (2) Structured output carries an explicit "not in the documents" status, so declining is a normal outcome. (3) A grader model (gpt-4.1-mini) lists every answer statement that the sources don't support. The writer (gpt-4.1-nano) gets one revision with that list; if it still fails, the answer is shown as "unverified" and queued for review. (4) NeMo Guardrails output rails block personal data and instructions injected from documents. *Alternatives:* prompt instructions alone cannot detect a hallucination after it happens, and nano as the grader flagged most correct answers, so the checker runs on the stronger model.

3. Results and error analysis

The held-out set has 74 questions across 10 types, including law vs. policy, multi-document, identifier, version, conflicting-source, vague, unanswerable and off-topic questions. It has a fixed stratified split of 27 development and 47 test questions; all tuning used the development split. On the 47 test questions the full agent reaches: correct outcome (answer / clarify / decline) **0.936**; key facts correct **0.974**; faithfulness, the share of answer statements supported by the retrieved context (judged by gpt-4.1-mini), **0.982** over all answers and **0.994** excluding the 4% shown to the user as "unverified" (the gap is the errors the grounding check caught); gold passages found by the agent's searches 0.932, with 1.7 searches and 8.1 s per question.

A safety suite passed 26 of 26 checks: jailbreak, personal-data and off-topic inputs blocked; greetings allowed; no restricted document retrieved for an unauthorized role; as-of dates; follow-up memory; and the hidden injection ignored.

Query analysis. "How often is refresher training required?" names no topic. The agent calls Clarify ("For which training topic or role...?") instead of guessing. "How do I reset my VPN password?" is refused by the input guardrail before any search. Messages that need no documents ("Hi", "thanks", "what can you do?") go to a SmallTalk tool that returns a fixed, pre-written reply, so no ungrounded claim can be produced and nothing needs verifying.

Failures. The cheap writer twice attributed a policy rule or a neighbouring paragraph to the wrong source; the grounding check caught one (shown as "unverified"), the other ended as "not in the documents". One correct answer was labelled "out of scope", and the guardrails refused two legitimate requests (false positives). Conflicting-source and version questions remain hardest for retrieval, because the informal e-mail or old policy matches the wording best; the current-policy-first rule is enforced at generation, not in search. The evaluation is small (wide intervals), the internal documents are cleaner than real ones, and one author wrote both corpus and questions.

4. Production and limitations

Knowledge freshness. LangChain's Indexing API hashes every chunk and records its source file in SQLite, so each sync embeds only new or changed text and deletes chunks of edited or removed files. Measured: a new document embedded 2 chunks; a one-line edit re-embedded 1 chunk and deleted 1, while 966 were skipped; a deletion removed 2 chunks; no full re-embedding is needed. Superseded versions stay indexed for history, are labelled in each chunk, and are filtered out by as-of queries.

1,000 concurrent queries. Search is sub-millisecond (exact FAISS over 965 chunks). The bottleneck is 6–9 LLM calls per question (about 900 calls/s at 8 s per question), bounded by provider rate limits. Fixes: asynchronous workers with raised quotas or several model deployments; the input guardrail run in parallel with the first search; the built-in response cache; a hosted cross-encoder instead of LLM re-ranking (up to 3 fewer calls per question); and Postgres memory plus a shared index (or pgvector) so several servers can run.

Limitation before real deployment. The user's role is currently chosen in the interface. It must come from single sign-on identity claims, because access control is only as strong as that input. Scanned PDFs would also need optical character recognition (OCR).